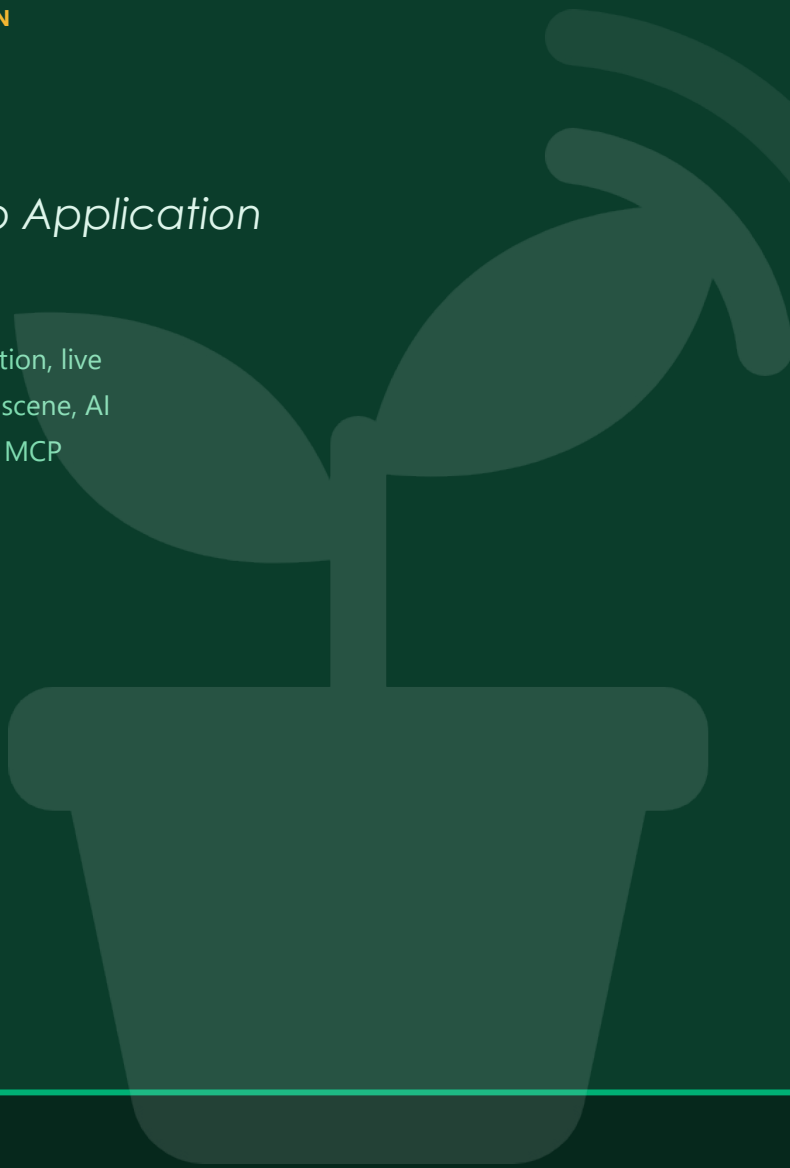
SmartPot

COMPONENT TECHNICAL DOCUMENTATION

SmartPot-Web

Next-Generation Progressive Web Application

Installable SolidJS PWA without Virtual DOM: overview dashboard, native SVG charts, conditional ETag revalidation, live SSE streaming, real and virtual crops, reactive SVG crop scene, AI assistant, control, continuous learning, alerts, Telegram, MCP connections, dark mode, and full English/Spanish i18n.



DOCUMENT / DOCUMENTO VERSION / VERSIÓN
SmartPot-Web Documentation

TEAM / EQUIPO
1.1 · November 2026

PROJECT / PROYECTO
SmartPotTech smartpot.app

English | Español

CONTENTS · CONTENIDO

Contents / Contenido

1. Purpose
2. Component Architecture
3. Creating a Crop
4. Reactive Crop Illustration and Native SVG Charts
5. Session, Conditional ETag Cache, Internationalization, and Security
6. Configuration
7. Testing
8. Operations

PART I

Component Overview

SECTION 01

Purpose

Document Sheet

FIELD	VALUE
Project	SmartPot · smartpot.app
Component	SmartPot-Web
Version	1.1 · November 2026
Scope	Reactive SolidJS architecture, screens, native SVG charts, crop creation, live SVG crop scene, cross-tab `HttpOnly` cookie session + CSRF, `ETag` cache, bilingual i18n (`en`/`es`), dark mode, web security, configuration, and tests
Platform Documentation	Migration Documentation, Technical Documentation, and Project Journey
Maintenance	Generated from `docs/` in this repository using `.github/docs/tools`; updated alongside component changes

In Plain Terms

`SmartPot-Web` is the Progressive Web Application (PWA) of SmartPot, built with **SolidJS + TypeScript + Vite**. It displays every crop live, compares sensor readings against the ideal range for its species, lets users create real or virtual crops, send actuator commands, and inspect what the AI assistant diagnoses and learns, while minimizing bundle size and browser work via fine-grained reactivity without a Virtual DOM, native SVG charts, live SSE streaming with LTTB downsampling, conditional `ETag` revalidation, dark mode, and full bilingual support (**English and Spanish**).

SCREEN	CAPABILITIES
Public Landing	Product overview, features, supported species, and FAQ; search-engine indexable
Overview Dashboard	Health ranking of every crop, single-metric comparison in native SVG, latest readings table, and AI fleet analysis across the account
My Crops	Crop cards with kind, species, form, device status, and health; step-by-step creation wizard

SCREEN	CAPABILITIES
Crop Detail	Live SVG illustration of the crop in its placement above all tabs: summary, AI assistant with placement advice, switch controls, history, device (real) or simulation (virtual), settings
General Control & Actions	Automatic mode and bulk commands; AI-suggested fleet actions and command history
Learning, Alerts & Profile	Continuous AI learning metrics, notifications, Applications (Telegram), personal MCP connections (`spt_...`), profile data, and password

SECTION 02

Component Architecture

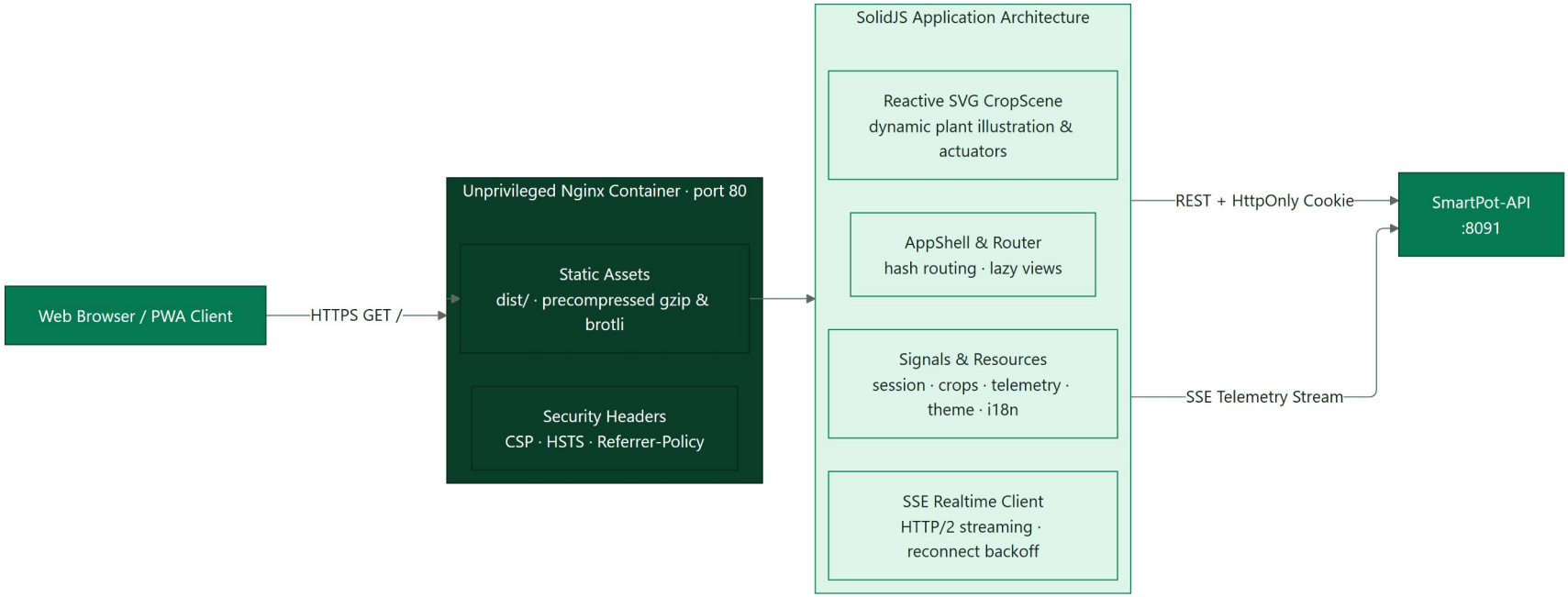


Figure / Figura 1 · *SmartPot-Web Internal Architecture*

Each domain feature lives in `src/features/<feature>` with its pages, components, and tests. `lib/api` is the single gateway to the API: it sends requests with `credentials: "include"` and `X-CSRF-Token`, manages the conditional `ETag` cache, maps errors to localized `ApiError` instances (`en`/`es`), and declares response types. `useResource` loads and refreshes SolidJS signals while the tab is visible.

PART II

Crops and Live Scene

SECTION 03

Creating a Crop

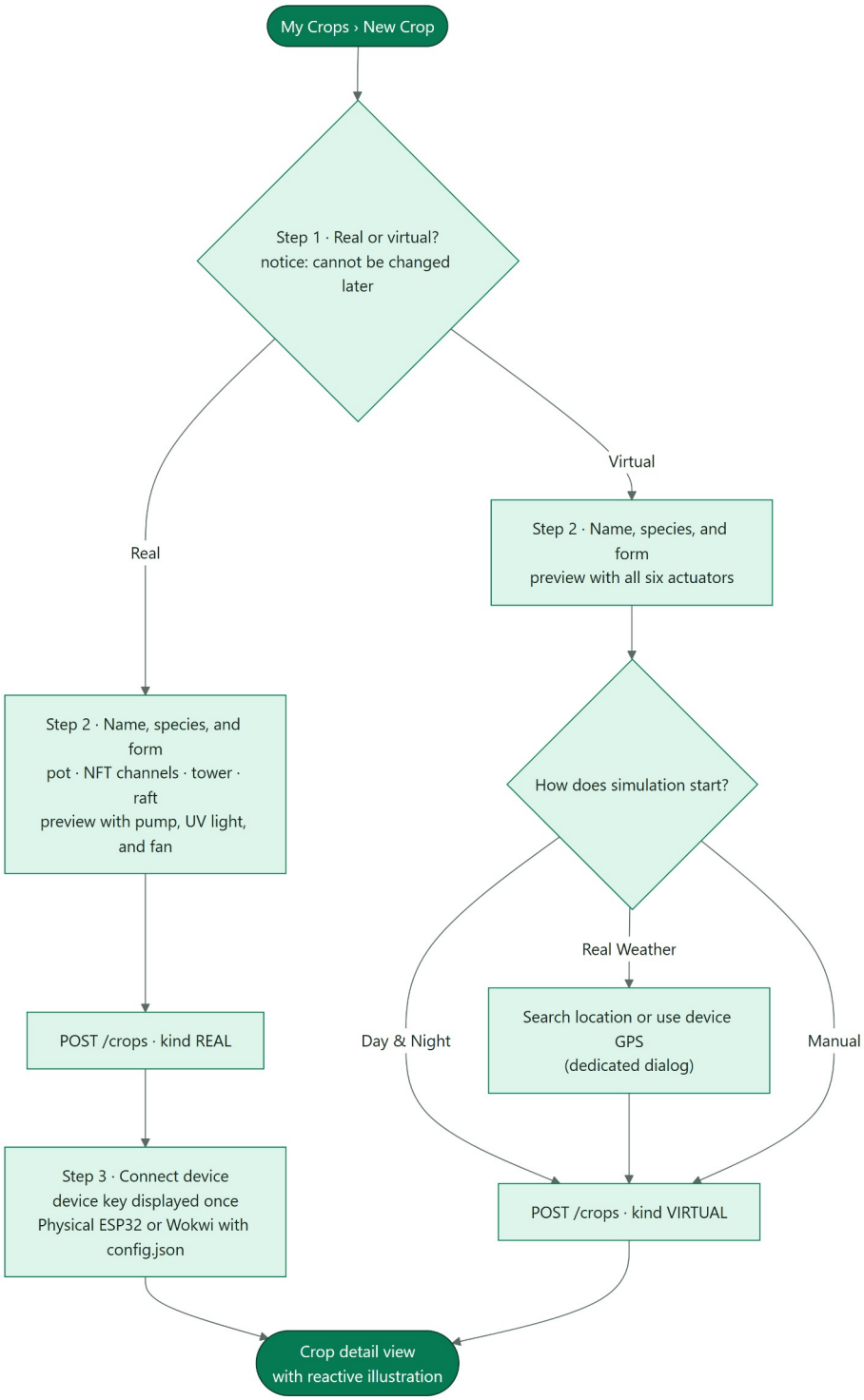


Figure / Figura 2 · *Crop Creation Flow*

DECISION	DETAIL
Kind	Real (an ESP32 running the firmware, physical or in Wokwi) or Virtual (simulated by SmartPot). Cannot be changed later; Settings explains how to create another
Form	Pot, NFT tubes, vertical tower, or floating raft; can be changed in Settings and updates the live illustration
Placement	Indoor or outdoor, sun exposure, and city/coordinates; editable in Settings
Preview	Live SVG crop scene with the selected species, form, and initial actuators
Real	Displays the device key once along with the wiring guide, firmware repository, and ready-to-copy `config.json` with WiFi and embedded Broker CA (`/ca.crt`)
Virtual	Day/night cycle, real local weather, or manual sliders; opens the crop with full simulation controls in the Simulation tab

SECTION 04

Reactive Crop Illustration and Native SVG Charts

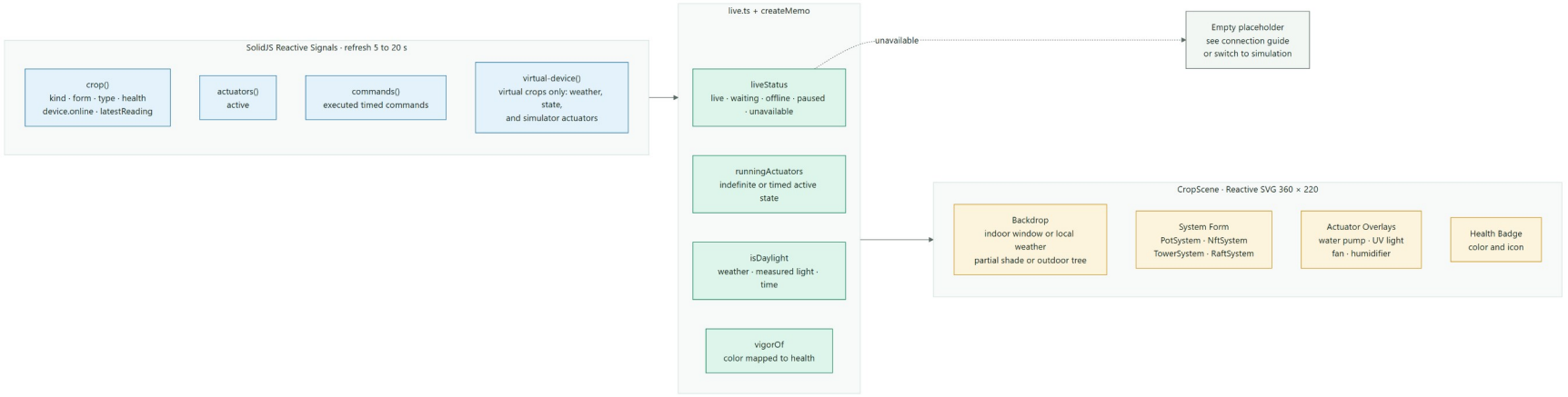


Figure / Figura 3 · *Dynamic Scene Rendering Engine*

ELEMENT	VISUAL REPRESENTATION
Pot	Reservoir with water pump, drip tube feeding the substrate, and nutrient/pH dosers on the lid
NFT Tubes	Three channels with four plants each, inlet manifold, and return line; nutrient film flows when the pump is active
Vertical Tower	Column with staggered pockets; pump lifts the solution and droplets cascade inside
Floating Raft	Deep-water basin with floating raft, submerged roots, and rising air-stone bubbles
UV Grow Light	Overhead LED bar; illuminates the canopy when active
Fan & Humidifier	Spinning blades with airflow lines; rising mist plume
Plants	Species-accurate geometry (rosette lettuce, broadleaf spinach, paired basil leaves, staked tomato with flowers/fruit, strawberry, bell pepper) colored by health vigor

SECTION 05

Session, Conditional ETag Cache, Internationalization, and Security

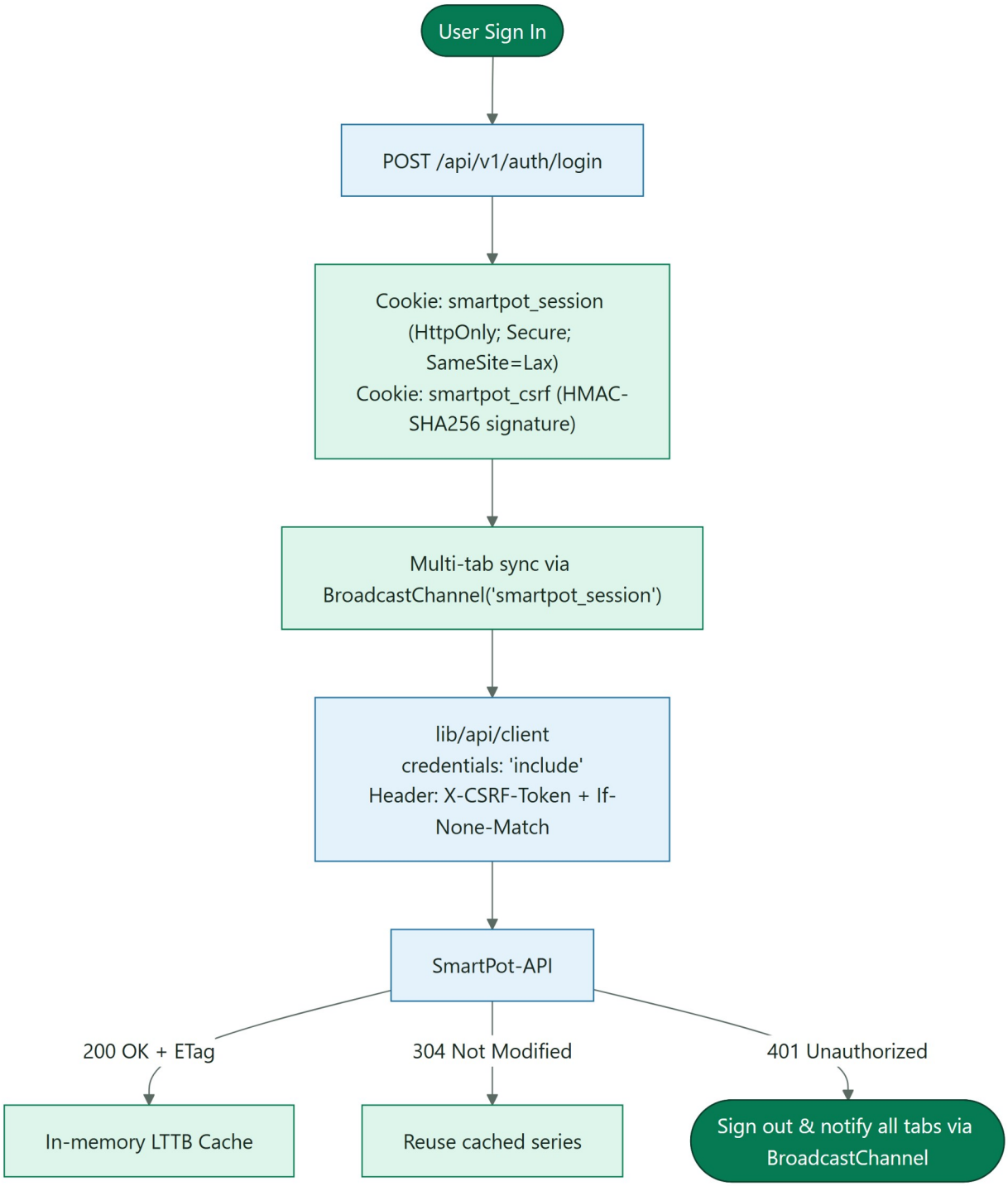


Figure / Figura 4 · *Session Lifecycle and Offline Storage*

CONTROL	DETAIL
CSP & CORP	`script-src 'self'`, `style-src 'self'`, `connect-src` restricted to the API, and `Cross-Origin-Resource-Policy: same-origin`
Security Headers	`X-Frame-Options: DENY`, `nosniff`, `Referrer-Policy`, and `Permissions-Policy` (geolocation restricted to same-origin and explicit user action)
`ETag` Revalidation	Client sends `If-None-Match` on `GET` requests; `SmartPot-API` replies `304 Not Modified` when unchanged and invalidates cache on any write
Session & CSRF	`HttpOnly` cookie (`smartpot_session`) + signed double-submit CSRF token (`smartpot_csrf` / `X-CSRF-Token`), synchronized across tabs via `BroadcastChannel`
Bilingual (`en`/`es`)	Reactive dictionary (`src/lib/i18n/en.ts`, `es.ts`) and dynamic API/AI message translation (`serverText.ts`) so the entire UI switches languages instantaneously
Device Key	Displayed only once upon real crop creation or key rotation

SECTION 06

Configuration

VARIABLE	LOCATION	PURPOSE
`VITE_API_URL`	`.env` in development	Target API URL for `bun run dev`
`API_URL`	Container runtime	Injected at startup into `/config.js` and the Nginx CSP header so a single image runs in any env
`MCP_URL`	Container runtime	Optional public URL of `SmartPot-MCP` shown when generating MCP client snippets in Profile

SECTION 07

Testing

Run `bun run lint`, `bun run typecheck`, `bun run test`, and `bun run build`. The Vitest and `@solidjs/testing-library` suite covers the HTTP client, `ETag` revalidation (`304 Not Modified`), `HttpOnly` session and CSRF flow, form validations, login, crop components, actuator switches, AI insights, continuous learning, Telegram channels, MCP connections, live SVG scenes, dark mode, `es`/`en` internationalization, and SEO/PWA compliance.

SECTION 08

Operations

TASK	COMMAND / MECHANISM
Docker Image	`ghcr.io/smartpottech/smartpot-web`: compiled with Bun Alpine and served by unprivileged Nginx
demo-beta	Runs as `web-smartpot` in `.github/docker/demo-beta/compose.yaml`
Icons	`bun run icons` regenerates all PWA PNG icons and Open Graph images from SVG sources
Browser Cache	Service worker version bump replaces the cached app shell on reload